

List

24 July 2026 11:37

The data type list is an ordered sequence which is mutable and made up of one or more elements. Unlike a string, which consists of only characters, a list can have elements of different data types such as integer, float, string, tuple, or even another list. A list is a very useful to group together elements of mixed data types. Elements of a list are enclosed in brackets. And are separated by comma(,). Like string indices, list indices also starts from zero. Example:

```
#list1 is the list of 10 even numbers
```

```
list1 = [2,4,6,8,10,12,14,16,18,20]
```

```
print(list1)
```

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

```
# list2 is List of vowels.
```

```
list2 = ['a','e','i','o','A','E', 'I','O','U']
```

```
print(list2)
```

```
['a', 'e', 'i', 'o', 'A', 'E', 'I', 'O', 'U']
```

```
#list3 is list of mixed values
```

```
list3 = [100.23, 500, 'Hello',[1,2,3], 5+2j]
```

```
print(list3)
```

```
[100.23, 500, 'Hello', [1, 2, 3], (5+2j)]
```

```
#list4 is list of lists.
```

```
list4 = [['Physic', 101], ['Chemistry', 202], ['Maths', 303]]
```

```
print(list4)
```

```
 [['Physic', 101], ['Chemistry', 202], ['Maths', 303]]
```

Accessing Elements in a List

The elements of the list are exist in the same way as characters are accessed in the string.

```
print(list4[0]) # Returns first element of list4.
```

```
['Physic', 101]
```

```
print(list4[-1]) # Returns last element of list 4.
```

```
['Maths', 303]
```

```
print(list3[0])
```

```
100.23
```

```
print(list3[-1])
```

```
(5+2j)
```

```
list1
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
list1[4]
10
```

List are mutable

In Python, lists are mutable. It means that the contents of the list can be changed after it has been created.

```
list1
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
list1[4]
10
list1[4] = list1[4]*2
list1[4]
20
list1
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20]
```

List Operations

The data type list allows manipulation of its contents through various operations as shown below:

1. **Concatenation:** Python allows us to join two or more list using the concatenation operator depicted by the symbol plus(+). For example:

```
list1
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20]
list5 = [13, 15, 17, 19]
list6 = list1 + list5
list6
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
```

2. **Repetition:** Python allows us to replicate a list using repetition operator depicted by symbol *. For example:

```
list5 = [13, 15, 17, 19]
list7 = list5*3
list7
[13, 15, 17, 19, 13, 15, 17, 19, 13, 15, 17, 19]
```

3. **Membership:** Like strings, the membership operators 'in' and checks if the element is present in the list and returns True, else returns False. For example:

```
list6
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
11 in list6
False
```

```
12 in list6
```

```
True
```

The '**not in**' operator returns True if the element is not present in the list, else it returns False.

For example:

```
list6
```

```
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
```

```
11 not in list6
```

```
True
```

```
12 not in list6
```

```
False
```

Slicing: Like strings, the slicing operation can also be applied to list.

For example:

```
list6
```

```
[2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
```

```
list6[:5]
```

```
[2, 4, 6, 8, 20]
```

```
list6[3:5]
```

```
[8, 20]
```

```
list6[-6:-2]
```

```
[18, 20, 13, 15]
```

```
list6[-6:-2:-2]
```

```
[]
```

```
list6[-6:-2:2]
```

```
[18, 13]
```

```
list6[-2:-6:-2]
```

```
[17, 13]
```

```
#Reversing the list
```

```
list6[::-1]
```

```
[19, 17, 15, 13, 20, 18, 16, 14, 12, 20, 8, 6, 4, 2]
```

```
#Alternate values
```

```
list6[::2]
```

```
[2, 6, 20, 14, 18, 13, 17]
```

Traversing a list

We can access each element of the list or traverse or list using a for loop or a while loop.

A. List traversal using for loop

```
count = 0
```

```
list6 = [2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
```

```
for e in list6:
```

```
    if e % 3 == 0 or e % 5 == 0:
```

```

        print(e, end = ' ')
        count += 1
    print(f'\nnumber of values divisible by 3 or 5 is {count}')
```

Output:

```

6 20 12 18 20 15
number of values divisible by 3 or 5 is 6
```

B. List traversal using while loop

```

count = 0
list6 = [2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19]
index = 0
while index < len(list6):
    if list6[index] % 3 == 0 or list6[index] % 5 == 0:
        print(list6[index], end = ' ')
        count += 1
    index += 1
print(f'\nnumber of valueindex += 1s divisible by 3 or 5 is {count}')
```

Output:

```

6 20 12 18 20 15
number of values divisible by 3 or 5 is 6
```

List methods and built in Functions

The data type List has several built in methods that are useful in programming.

Built-in function for list manipulation

Method	Description	Example
len()	Returns the length of the list passed as the argument.	<pre>list6 = [2, 4, 6, 8, 20, 12, 14, 16, 18, 20, 13, 15, 17, 19] len(list6)</pre> 14
list()	Create an empty list if no argument is passed. Creates a list if a sequence is passed as an argument.	<pre>li1 = list() li1 [] len(li1)</pre> 0

		<pre>li1 = list('aeiou') li1 ['a', 'e', 'i', 'o', 'u'] len(li1) 5</pre>